Question 1:

Andrew Branum – 5291638

Question 2:

Yes

Question 3:

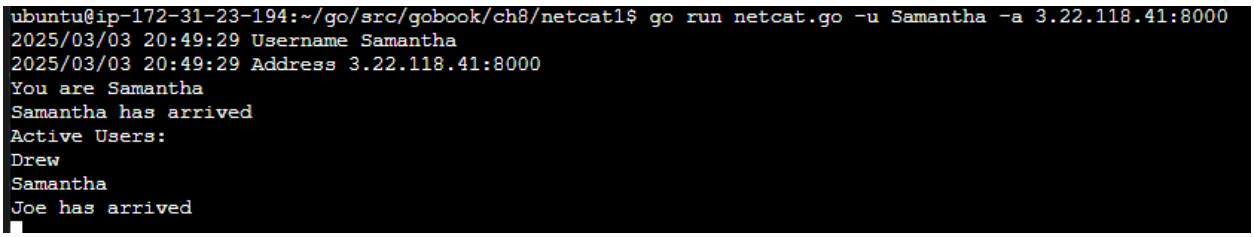
Part 1: To complete part 1, I used the flag package to capture the command line arguments as I did in the first project. After capturing the address and the user's username, I modified the Dial parameters to contain the newly entered address as I did in the first project. I did the same for the chat.go code and changed the Listen parameters. Sending the username over was tricky at first, but once I did enough research on socket connections I found that the command line basically writes and reads from a file so I opted to use io.WriteString to write the username to the file by basing the function off of the provided mustCopy() function. The sendUsername function is called right before the mustCopy function with conn and username as parameters. To capture the username in chat.go, I moved the instantiation of the new scanner above the logic that sends messages into the channels to be able to capture the username correctly. I simply used the Scanner to scan for any text, trimmed the text's whitespace, and assigned it to a variable "who". To capture all the clients and their usernames, I chose to move the clients map to the global scope and changed it from map[client]bool to map[string]chan<-string where the key is the client's username, and the value is the client's channel address. These are assigned within the handleConn function and removed within the broadcaster function when the client leaves the chatroom.

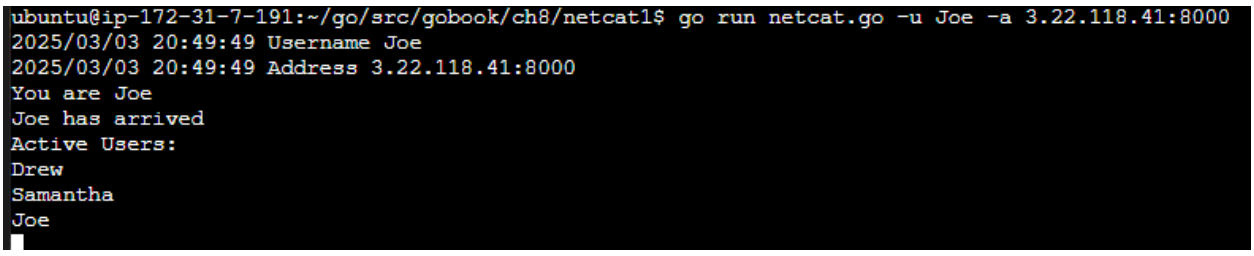
Part 2: Since I moved the client map to the global scope, broadcasting the names of the current users to new clients was straightforward. Since the usernames and channels are added to the map upon connection and globally accessible, the broadcast function simply iterates over the clients map and sends the usernames across the cli channel. Since I changed the client map, I also had to change the leaving functionality within the broadcaster function. Instead of simply deleting the client from the map, the program iterates over the clients map until the channel within the map matches the channel that is leaving and deletes the user from the map based on the channel and username. The loop is then broken, and the connection is closed on that channel. I also had to modify the portion of the broadcaster that sends messages to all other clients because of the changed client map. I simply had to iterate over the values of the client map rather than the keys to extract the channels to send messages to.

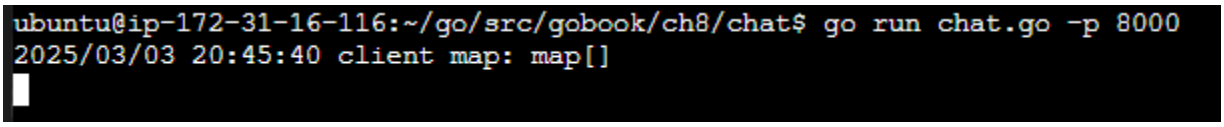
Part 3: To implement private messages, I took some inspiration from Minecraft such that sending a private message to someone follows the format of /username message. This format allows for a simple string split to parse the message and username. I used the Scanner the same way it was used in the original code but checked to see if messages started with "/" before continuing. If no "/" is detected the message is sent normally. If a "/" is detected, the text is split using SplitN to capture the respective substrings: username and message. Once the substrings are captured, the username that is receiving the private message is used to find their respective channel value and it is assigned to a "channel" variable. The message is then sent across that channel in the form DM (username who sent the message): Message.

Screenshots:

Joining Server:







Sending Messages:

```
ubuntu@ip-172-31-31-153:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Drew -a 3.22.118.41:8000
2025/03/03 20:49:03 Username Drew
2025/03/03 20:49:03 Address 3.22.118.41:8000
You are Drew
Drew has arrived
Active Users:
Drew
Samantha has arrived
Joe has arrived
howdy, I'm Drew
Drew: howdy, I'm Drew
Samantha: Hey Drew, I'm Samantha
Joe: I'm Joe!
```

```
ubuntu@ip-172-31-7-191:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Joe -a 3.22.118.41:8000
2025/03/03 20:49:49 Username Joe
2025/03/03 20:49:49 Address 3.22.118.41:8000
You are Joe
Joe has arrived
Active Users:
Drew
Samantha
Joe
Drew: howdy, I'm Drew
Samantha: Hey Drew, I'm Samantha
I'm Joe!
Joe: I'm Joe!
```

```
ubuntu@ip-172-31-23-194:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Samantha -a 3.22.118.41:8000
2025/03/03 20:49:29 Username Samantha
2025/03/03 20:49:29 Address 3.22.118.41:8000
You are Samantha
Samantha has arrived
Active Users:
Drew
Samantha
Joe has arrived
Drew: howdy, I'm Drew
Hey Drew, I'm Samantha
Samantha: Hey Drew, I'm Samantha
Joe: I'm Joe!
```

Private Messages:

```
ubuntu@ip-172-31-31-153:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Drew -a 3.22.118.41:8000
2025/03/03 20:49:03 Username Drew
2025/03/03 20:49:03 Address 3.22.118.41:8000
You are Drew
Drew has arrived
Active Users:
Drew
Samantha has arrived
Joe has arrived
howdy, I'm Drew
Drew: howdy, I'm Drew
Samantha: Hey Drew, I'm Samantha
Joe: I'm Joe!
/Samantha Who is this Joe guy?
DM (Samantha): No idea...
```

```
ubuntu@ip-172-31-23-194:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Samantha -a 3.22.118.41:8000
2025/03/03 20:49:29 Username Samantha
2025/03/03 20:49:29 Address 3.22.118.41:8000
You are Samantha
Samantha has arrived
Active Users:
Drew
Samantha
Joe has arrived
Drew: howdy, I'm Drew
Hey Drew, I'm Samantha
Samantha: Hey Drew, I'm Samantha
Joe: I'm Joe!
DM (Drew): Who is this Joe guy?
/Drew No idea...
```

Leaving Server:

```
ubuntu@ip-172-31-7-191:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Joe -a 3.22.118.41:8000
2025/03/03 20:49:49 Username Joe
2025/03/03 20:49:49 Address 3.22.118.41:8000
You are Joe
Joe has arrived
Active Users:
Drew
Samantha
Joe
Drew: howdy, I'm Drew
Samantha: Hey Drew, I'm Samantha
I'm Joe!
Joe: I'm Joe!
I'm outta here
Joe: I'm outta here
^Csignal: interrupt
ubuntu@ip-172-31-7-191:~/go/src/gobook/ch8/netcat1$
```

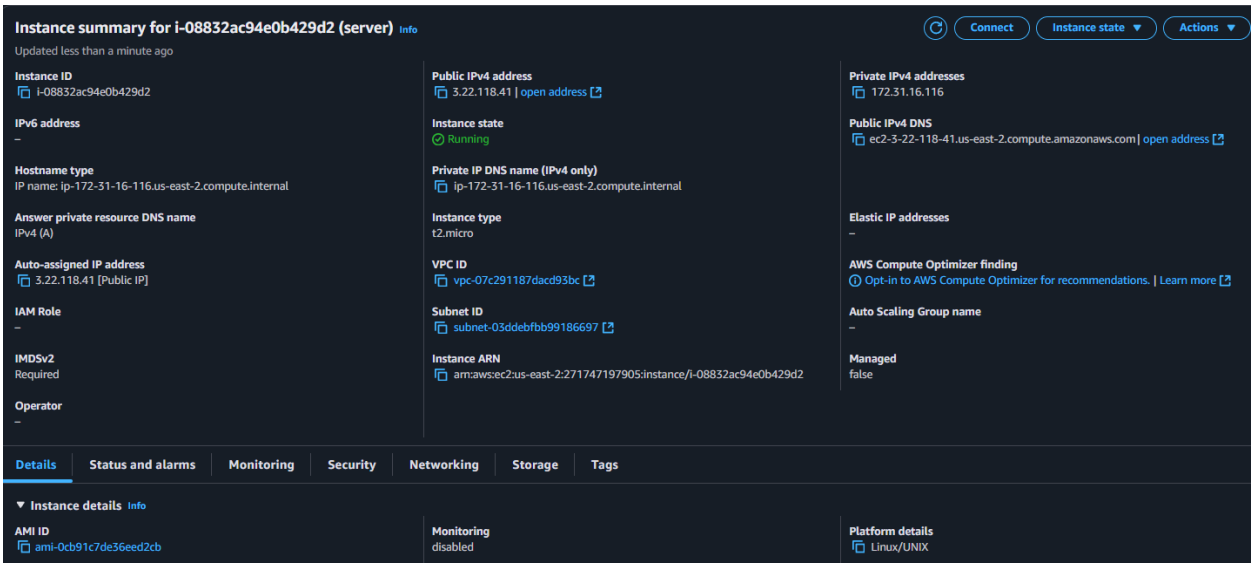
```
ubuntu@ip-172-31-31-153:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Drew -a 3.22.118.41:8000
2025/03/03 20:49:03 Username Drew
2025/03/03 20:49:03 Address 3.22.118.41:8000
You are Drew
Drew has arrived
Active Users:
Drew
Samantha has arrived
Joe has arrived
howdy, I'm Drew
Drew: howdy, I'm Drew
Samantha: Hey Drew, I'm Samantha
Joe: I'm Joe!
/Samantha Who is this Joe guy?
DM (Samantha): No idea...
Joe: I'm outta here
Joe has left
```

Joining after someone left (check map):

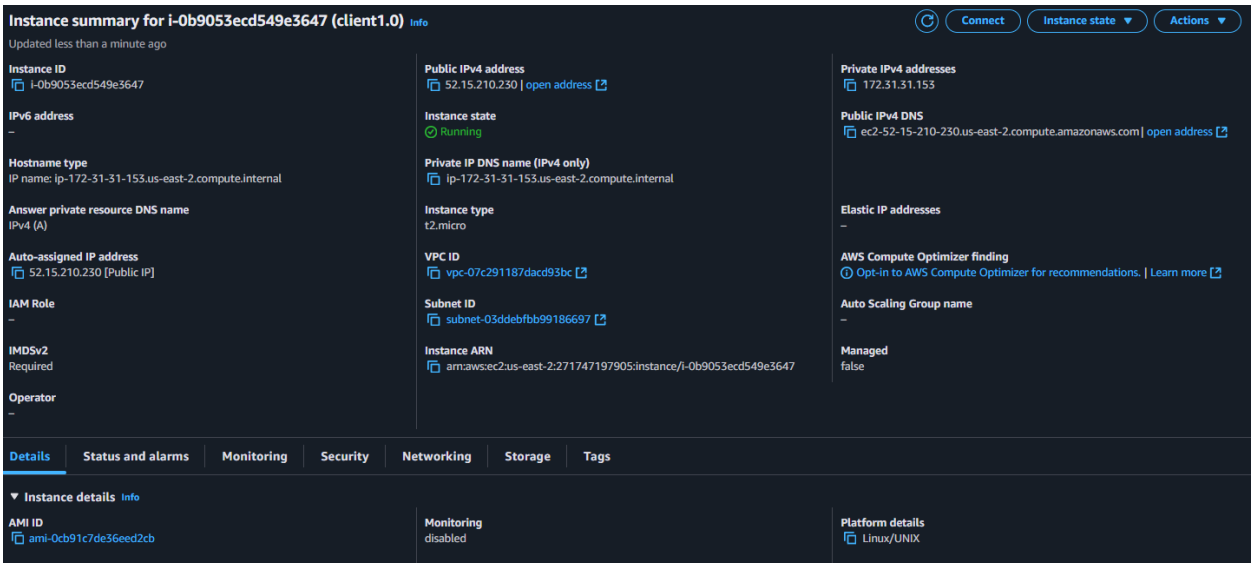
```
ubuntu@ip-172-31-7-191:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Jim -a 3.22.118.41:8000
2025/03/03 20:55:57 Username Jim
2025/03/03 20:55:57 Address 3.22.118.41:8000
You are Jim
Jim has arrived
Active Users:
Drew
Samantha
Jim
```

```
ubuntu@ip-172-31-31-153:~/go/src/gobook/ch8/netcat1$ go run netcat.go -u Drew -a 3.22.118.41:8000
2025/03/03 20:49:03 Username Drew
2025/03/03 20:49:03 Address 3.22.118.41:8000
You are Drew
Drew has arrived
Active Users:
Drew
Samantha has arrived
Joe has arrived
howdy, I'm Drew
Drew: howdy, I'm Drew
Samantha: Hey Drew, I'm Samantha
Joe: I'm Joe!
/Samantha Who is this Joe guy?
DM (Samantha): No idea...
Joe: I'm outta here
Joe has left
Jim has arrived
```

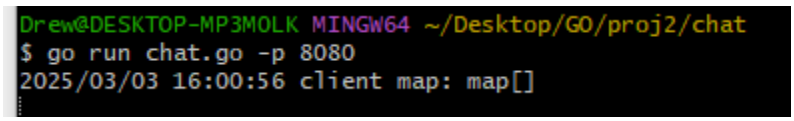
Server EC2 Setup:

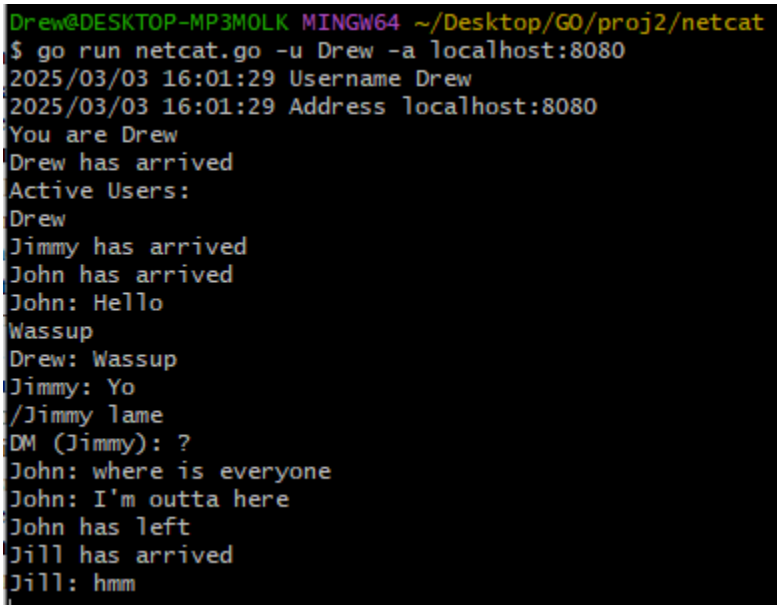


Client EC2 Instance Example:



Local Example:





```
$ go run netcat.go -u John -a localhost:8080
2025/03/03 16:02:24 Username John
2025/03/03 16:02:24 Address localhost:8080
You are John
John has arrived
Active Users:
Drew
Jimmy
John
Hello
John: Hello
Drew: Wassup
Jimmy: Yo
DM (Jimmy): secret message
/Jimmy why
where is everyone
John: where is everyone
I'm outta here
John: I'm outta here
exit status 0xc000013a
```

```
Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/GO/proj2/netcat
$ go run netcat.go -u Jill -a localhost:8080
2025/03/03 16:04:11 Username Jill
2025/03/03 16:04:11 Address localhost:8080
You are Jill
Jill has arrived
Active Users:
Jimmy
Jill
Drew
hmm
Jill: hmm
```

```
Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/GO/proj2/netcat
$ go run netcat.go -u Jimmy -a localhost:8080
2025/03/03 16:01:56 Username Jimmy
2025/03/03 16:01:56 Address localhost:8080
You are Jimmy
Jimmy has arrived
Active Users:
Drew
Jimmy
John has arrived
John: Hello
Drew: Wassup
Yo
Jimmy: Yo
/John secret message
DM (John): why
DM (Drew): lame
/Drew ?
John: where is everyone
John: I'm outta here
John has left
Jill has arrived
Jill: hmm
```